Technical University of Munich

Exercises Rechnereinführung WS 2026/27

Adrian Stein – Thomas Lindae – Ari Breitkreuz

---

2 – Linux (Part 2)

Exercises marked with a (*) are bonus tasks and should only be done if there is enough time left.

2.1 Package Management

This section is about practicing the usage of package managers. Therefore, we will install four small commands. Always check before installing a command to see if it has already been installed. Just try to execute it once. This will tell you if the command is already installed; if not, the resulting error message might give you instructions on installing the command with your package manager.

2.1.1

Please install the following commands:

1. `fortune`
2. `cowsay`
3. `sl`
4. `lolcat`

Note: Users of

- *Fedora* use the package manager `dnf`.
- *Ubuntu* use the package manager `apt`.
- *Windows* use the *WSL*. The default Linux distribution of the *WSL* is *Ubuntu*. So you also use the package manager `apt` (you may need to run `sudo apt update` before being able to install packages).
- *MacOS* use Homebrew (`brew`). You should have already installed this package manager during the first exercise. You can find help for the installation of individual commands if you search the Internet for e.g. "Homebrew fortune".

Solution

Fedora:

- `sudo dnf install fortune-mod`
- `sudo dnf install cowsay`
- `sudo dnf install sl`
- `sudo dnf install lolcat`

Ubuntu resp. *Windows* (*WSL*):

- `sudo apt install fortune-mod`
- `sudo apt install cowsay`
- `sudo apt install sl`
- `sudo apt install lolcat`

MacOS:

- `brew install fortune`
- `brew install cowsay`
- `brew install sl`
- `brew install lolcat`

2.2 Fun with Programs

1. Execute `lolcat` without arguments. What does this program do?
   (*Hint: You can terminate almost every program with Ctrl+C*)
2. Ececute `sl` without arguments. What typo is it supposed to get rid of?
3. Create a ranbow train!
4. Create a rainbow cow that tells you a fortune!
5. Can you also use the text editor *nano* in rainbow colors?
6. (*) Rename a file `a.txt` to `b.txt`. Create an empty file `b.txt` if renaming failed (because `a.txt` might not exist).

7. (*) Redirect some output to `/dev/null`. Print the contents of `/dev/null`. What do you notice? Use your new knowledge to reduce the command output created in 6 in the event of an error (you may need to search the web on how to do this).

Solution

1. Rainbow coloring effect for text console display.
2. It punishes typing `sl` instead of `ls`.
3. `sl | lolcat`
4. `fortune | cowsay | lolcat`
5. `nano | lolcat`
6. `mv a.text b.txt || touch b.txt`
7. `/dev/null` removes any recieved input.
   `mv a.txt b.txt 2> /dev/null || touch b.txt`

2.3 Access Control

2.3.1 `chmod`

First skim through the `man` page for `chmod` and try the assignment of permissions with this online tool. In what sense is the representation of permissions related to the octal system?

Answer the following questions:

1. Specify the numbers (from 0 to 7) that correspond to the following rights:
   a) write and execute
   b) read and execute
   c) rw
   d) x
   e) read
   f) no permissions
2. Specify a `chmod` command to take away your own read and write permissions to a file `dummy.txt`. The execution rights should not be changed.

3. Specify a `chmod` command which gives the assigned group all permissions to a file `file`. Everyone else (including the owner) should not get any permissions.

Solution

Since we always want to assign three types of "binary" permissions (*read*, *write*, *execute*) per user/group/other, it makes sense to represent them in the octal system (since $2^3 = 8$. We can thus encode all three permissions in exactly one digit).

1. a) 3
   b) 5
   c) 6
   d) 1
   e) 4
   f) 0
2. `chmod u-rw dummy.txt`
3. `chmod 070 file`

2.4 SSH

In this section, we connect to the "Rechnerhalle" and set up a few things.

2.4.1 Establishing a Connection

Use the command `ssh <rechnerkennung>@lxhalle.in.tum.de` to connect to the "Rechnerhalle" (you will probably need to verify the fingerprints of the "Rechnerhalle").

2.4.2 SSH-Keys

SSH keys are used to make connecting via SSH simpler. They allow quick and safe authorization.

In short, you have a "private key" and a "public key". As the names suggest, the private key is private and should *never* be moved to another machine. The public key on the other hand can be given to anyone and is only useful if you also have access to the corresponding private key.

The commands entered in the following sections can be quite long. Remember to use the *Tab* key to open options for autocompletion and that you can always cancel a command by using *Ctrl+C*.

- Generate a new key pair on your **local device** (either exit the ssh session or start a new terminal) with the command `ssh-keygen -t ed25519`. You can just confirm the file names by pressing *Enter* and do not need to change anything here (unless you want to, of course).
- Now the public key has to be transmitted to the computer hall. The easiest way to do this is by using the command `ssh-copy-id -i ~/.ssh/id_ed25519 <rechnerkennung>@lxhalle.in.tum.de`.[1]
- You will now be prompted to enter your password for the "Rechnerhalle". You may also have to write "yes" to confirm that you trust the connection.
- It should now be possible to connect to the Rechnerhalle without having to authorize yourself with the password of the "Rechnerhalle" (with the command `ssh <rechnerkennung>@lxhalle.in.tum.de`).

2.4.3 Change Password

Change your ITO credentials via Ucentral (**Important**: The changed password might be available a day later, so password authentication might not work for a day. Access via ssh is unaffected by a password change). Alternatively, you can also use the command `passwd` directly on the "Rechnerhalle". Use a secure password which you can remember (or better yet: use a password manager like Bitwarden or KeePassXC). Having to reset the password would be a hassle…

2.5 Text Editing in the Terminal

In this task you will use the terminal text editor *nano*[2] and *vim*[3]. The basics of their usage have already been covered in the lecture.

You will now copy the *Super-Cat* shell script from the lecture (once with *nano* and once with *vim*). Without copy-paste, of course!

First, create an empty file called `supercat.sh` with `touch` (make the file executable with `chmod`). If you then switch editors, you can simply delete the old file or create a new one.

[1] In some cases, this easy way does not work and you have to explicitly copy the public key and paste it into file `~/.ssh/authorized_keys`.
[2] nano user's guide
[3] If you need Help you may run `vimtutor` within a terminal.

```sh
#!/bin/sh

echo "supercat!"
read -p "enter file globbing pattern: " glob

for file in *"$glob"; do
    if [ -f "$file" ]; then
        echo "> Content of ${file}:"
        cat "$file"
    fi
done
```

You can execute the file in the terminal with `./supercat.sh`.

We highly recommend to check out the following resources, if you are interested in *vim* (or better yet: neovim):

- Mastering the Vim Language
- Vim As Your Editor—Introduction
- 0 to LSP: Neovim RC From Scratch

2.6 Shell Scripting

1. Write a script for the **sh** shell that collects all files (in the *CWD*) ending in `.txt` and outputs the name and number of lines contained in each file (format example (per file): `foo.txt has 8 lines`)

   Hint: Determine how the output of `wc` changes when you provide the file as a CLI Arg or when sending the contents to *stdin*!

2. (*) Write a script for the **sh** shell that adds[4] up the number of lines in all files in the *CWD* that end in `.txt` and outputs the total as the result (format example: `Total lines in all .txt files: 9000`).

[4] Search the Internet for ways to add two shell variables.

Solution

1.
```sh
#!/bin/sh

for file in *.txt; do
    if [ -f "$file" ]; then
        lines=$(wc -l < "$file")
        echo "$file has $lines lines"
    fi
done
```

2.
```sh
#!/bin/sh

total=0

for file in *.txt; do
    if [ -f "$file" ]; then
        lines=$(cat "$file" | wc -l)
        total=$((total + lines))
    fi
done

echo "Total lines in all .txt files: $total"
```

2.7 Handling Large Files/Outputs

2.7.1 Goethe's Faust

For this task, you will find on Moodle a text file (`faust.txt`) of Goethe's "Faust. Der Tragödie erster Teil".

1. Try to print the file contents with `cat`. What do you notice?
2. To counteract the limitation of `cat`, open the file once with `more` and once with `less`.
3. Use `grep` to count the occurences of the substring "Mephisto" in the file `faust.txt`.
4. Use `grep` to count the occurences of the substring "Gretchen" in the file `faust.txt`.
5. Use `grep` to find a sentance containing the string "studiert".

Solution

1. The terminal history cannot display the entire text of the file. It is therefore not possible to read the beginning of "Faust".
2. `more faust.txt` and `less faust.txt`. The primary concern here is to have a way to view the content meaningfully in the terminal. `less` is generally better.
3. `grep -c "Mephisto" faust.txt`—"Mephisto" occurs 30 times.
4. `grep -c "Gretchen" faust.txt`—"Gretchen" occurs 21 times.
5. `grep "studiert" faust.txt`

2.7.2 `grep` and Pipes

The command `grep` is especially useful when another command produces a large and confusing output. We force such a situation by searching for "lib" with our package manager. This would list all possible libraries or "libre" (from "liberty") packages. We would trigger this search with `dnf search lib` on *Fedora* (or `apt search lib` on *Ubuntu*). Filter the output using `grep` to only display the results related to the audio format "Vorbis" (disregarding letter capitalization).

Solution

`dnf search lib | grep -i "vorbis"` resp. `apt search lib | grep -i "vorbis"`